PROBLEM SOLVING USING C PROGRAMMING LABORATORY

LAB MANUAL

IFETCE R - 2026

IFET COLLEGE OF ENGINEERING

[An Autonomous Institution]

DEPARTMENT OF INFORMATION TECHNOLOGY

SUBJECT CODE : 26CS1L01

YEAR/SEM : I/I

Prepared by:	*Verified by:*	*Approved by:*

DEPARTMENT VISION

- To transform individuals into ethical IT professionals, researchers, innovators, and entrepreneurs with the skills to excel globally in the dynamic field of Information Technology.

DEPARTMENT MISSION

- Delivering a comprehensive and dynamic IT curriculum that aligns with industry standards and emerging technologies.
- Providing state-of-the-art infrastructure and resources to support hands-on learning and real-world problem-solving.
- Fostering a culture of research, innovation, and continuous learning among students and faculty.
- Collaborating with industry partners to enhance students' practical skills and employability.
- Encouraging ethical practices and social responsibility in the application of Information Technology.

PROGRAMME OUTCOMES (PO)

On the completion of the program the graduates are expected to know or develop the abilities defined through the Program Outcomes.

The Program Outcomes of Information Technology Programme is given below.

PO1: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAMME SPECIFIC OUTCOMES (PSO)

The Information Technology Graduates will:

PSO 1: Comprehend mathematical and computational methodologies to solve problems across various domains.

PSO 2: Design and develop computer programs in the areas related to algorithms, system software, cloud computing, artificial intelligence, machine learning, big data analytics, Blockchain, cyber security, and networking, to efficiently create computer-based systems of varying complexities.

PSO 3: Apply standard engineering practices and strategies for software project development using open-source programming environments.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

PEO 1: Graduates will excel in their professional careers and/or higher education by acquiring knowledge in the core and applied domains of Information Technology.

PEO 2: Graduates will analyze real-life problems across diverse fields and provide technically sound solutions that are economically feasible and socially acceptable.

PEO 3: Graduates will exhibit professionalism, ethical behavior, communication skills, and teamwork in their profession, while adapting to current trends through lifelong learning.

101	26CS1L01	Problem Solving Using C Programming Laboratory	Category: Engineering Science Course			
			Semester: I			
			L	T	P	C
			0	0	4	2
Pre-requisites (if any)		Nil				

COURSE OBJECTIVES

The faculty will enhance the skills of the students to

- Familiarize with basic programming concepts and operator usage in C.
- Illustrate the use of conditional and iterative constructs to control program flow.
- Gain problem-solving ability using arrays, functions, and recursion.
- Develop students ability to apply string handling, pointers, parameter passing, and dynamic memory allocation concepts for efficient C programming..
- Enable students to design and develop C applications by applying file handling, structures, unions, and real-time mini project concepts.

COURSE OUTCOMES:

Upon completion of the course, the students will be able to

CO1: Write C programs utilizing operators, expressions, and bitwise manipulation techniques.
CO2: Develop solutions through decision-making constructs and iterative looping mechanisms.
CO3: Implement solutions using arrays, functions, and recursion techniques.
CO4: Apply string handling, pointers, parameter passing, and dynamic memory allocation concepts to develop efficient C programs.
CO5: Design C applications using file handling, structures, unions, and real-time mini project implementation.

CO – PO MAPPING:

POs / COs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CO1	3	3	3	3	2	-	-	1	1	-	2	3	2	1
CO2	3	3	3	3	2	-	-	1	1	-	2	3	3	2
CO3	3	2	1	2	2	-	-	-	1	-	2	3	3	2
CO4	3	2	1	2	2	-	-	-	1	-	2	3	3	3
CO5	3	3	3	3	2	1	1	1	1	1	2	3	3	3
Average	3	3	2	2	2	1	1	1	1	1	**2**	**3**	**3**	**2**

(3/2/1 indicates strength of correlation) 3-Strong, 2-Medium,1-Weak)

INDEX
TABLE OF CONTENTS

SL. NO	DATE OF EXPT.	NAME OF THE EXPERIMENT	PAGE NO	MARKS			STAFF INITIAL
				Viva (10)	Expt. (10)	Total (20)	
1.		DEVELOP SIMPLE C PROGRAMS USING OPERATORS AND EXPRESSIONS					
2.		ILLUSTRATE BITWISE OPERATIONS THROUGH C PROGRAMS					
3.		DEVELOP PROGRAMS USING BRANCHING STATEMENTS (DECISION-MAKING).					
4.		DEVELOP PROGRAMS USING LOOPING STATEMENTS (ITERATION).					
5.		DEMONSTRATE ARRAY OPERATIONS USING 1D AND 2D ARRAYS					
6.		IMPLEMENT FUNCTIONS AND RECURSIVE SOLUTIONS IN C					
7.		C PROGRAMS USING STRINGS AND STRING FUNCTIONS					
8.		DEVELOP A C PROGRAM TO DEMONSTRATE THE USE OF POINTERS AND PARAMETER PASSING TECHNIQUES ALONG WITH THE IMPLEMENTATION OF DYNAMIC MEMORY ALLOCATION					
9.		APPLY FILE HANDLING CONCEPTS IN C PROGRAMS ALONG WITH THE USE OF STRUCTURES AND UNIONS					

10.		DEVELOP A MINI PROJECT FOR A REAL-TIME APPLICATION USING C CONCEPTS					
Virtual Lab							
13.							
14.							
Content Beyond The Syllabus							
15.							
16.							
Total marks							

Ex No: 1a	**DEVELOP SIMPLE C PROGRAMS USING OPERATORS AND EXPRESSIONS**

AIM:

To write a C program to demonstrate the use of arithmetic, relational, and logical Operators to perform calculations and comparisons on integer values.

ALGORITHM:

1. Start the program.
2. Declare two integer variables a and b, and a variable choice.
3. Read two numbers from the user.
4. Display a menu containing different arithmetic and relational operations.
5. Ask the user to select an operation.
6. Use the switch statement to perform the selected operation
7. Display the result of the selected operation.
8. If the user selects an invalid option, display an appropriate message.
9. Stop the program.

PROGRAM:

```
#include <stdio.h>
int main()
{
   int a, b, choice, res;
   printf("===== OPERATORS AND EXPRESSIONS =====\n");
   printf("Enter the first number: ");
   scanf("%d", &a);
   printf("Enter the second number: ");
   scanf("%d", &b);
   printf("\n----- MENU -----\n");
   printf("1. Addition\n");
   printf("2. Subtraction\n");
   printf("3. Multiplication\n");
   printf("4. Division\n");
   printf("5. Modulus\n");
   printf("\nEnter your choice: ");
   scanf("%d", &choice);
   switch(choice)
   {
      case 1:
         res = a + b;
         printf("Result = %d", res);
         break;
      case 2:
         res = a - b;
         printf("Result = %d", res);
```

```
            break;
        case 3:
            res = a * b;
            printf("Result = %d", res);
            break;
        case 4:
            if(b != 0)
            {
                res = a / b;
                printf("Result = %d", res);
            }
            else
            {
                printf("Division by zero is not possible.");
            }
            break;
        case 5:
            if(b != 0)
            {
                res = a % b;
                printf("Result = %d", res);
            }
            else
            {
                printf("Modulus by zero is not possible.");
            }
            break;
        default:
            printf("Invalid choice.");
    }
    return 0;
}
```

OUTPUT

```
===== OPERATORS AND EXPRESSIONS =====
Enter the first number: 20
Enter the second number: 10

----- MENU -----
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus

Enter your choice: 1
Result = 30
```

RESULT

The C program for performing the use of arithmetic, relational, and logical Operators to perform calculations and comparisons on integer values has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
CO1	3	3	3	3	2	-	-	1	1	-	2		3	2	1

Ex No: 2	**BITWISE OPERATIONS**

AIM

To write a C program to demonstrate the use of Bitwise operations.

ALGORITHM

1. Start the program.
2. Declare three integer variables a, b, res and choice.
3. Read two integer values from the user.
4. Display the menu for bitwise operations.
5. Read the user's choice.
6. Perform the selected bitwise operation:
   - & – Bitwise AND
   - | – Bitwise OR
   - ^ – Bitwise XOR
   - ~ – Bitwise NOT
   - << – Left Shift
   - >> – Right Shift
7. Store the calculated result in the variable res.
8. Display the result.
9. If the user enters an invalid choice, display an appropriate message.
10. Stop the program

PROGRAM:

```
#include <stdio.h>

int main()
{
   int a, b, res, choice;
   printf("===== BITWISE OPERATIONS =====\n");
   printf("Enter the first number: ");
   scanf("%d", &a);
   printf("Enter the second number: ");
   scanf("%d", &b);
   printf("\n----- MENU -----\n");
   printf("1. Bitwise AND (&)\n");
   printf("2. Bitwise OR (|)\n");
   printf("3. Bitwise XOR (^)\n");
   printf("4. Bitwise NOT (~)\n");
   printf("5. Left Shift (<<)\n");
   printf("6. Right Shift (>>)\n");
   printf("\nEnter your choice: ");
   scanf("%d", &choice);

   switch(choice)
```

```
    {
        case 1:
            res = a & b;
            printf("Bitwise AND Result = %d", res);
            break;
        case 2:
            res = a | b;
            printf("Bitwise OR Result = %d", res);
            break;
        case 3:
            res = a ^ b;
            printf("Bitwise XOR Result = %d", res);
            break;
        case 4:
            res = ~a;
            printf("Bitwise NOT Result = %d", res);
            break;
        case 5:
            res = a << b;
            printf("Left Shift Result = %d", res);
            break;
        case 6:
            res = a >> b;
            printf("Right Shift Result = %d", res);
            break;
        default:
            printf("Invalid choice.");
    }
    return 0;
}
```

OUTPUT

```
===== BITWISE OPERATIONS =====
Enter the first number: 12
Enter the second number: 5

----- MENU -----
1. Bitwise AND (&)
2. Bitwise OR (|)
3. Bitwise XOR (^)
4. Bitwise NOT (~)
5. Left Shift (<<)
6. Right Shift (>>)

Enter your choice: 1
Bitwise AND Result = 4
```

RESULT:

The C program for performing the use of Bitwise Operations to perform calculations and comparisons on integer values has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
CO1	3	3	3	3	2	-	-	1	1	-	2		3	2	1

Ex No: 3	**DEVELOP PROGRAMS USING BRANCHING STATEMENTS (DECISION-MAKING)**

AIM

To write a C program to demonstrates the use of Develop programs using branching statements.

ALGORITHM

1. Start the program.
2. Declare integer variables a, b, choice, and res.
3. Read two numbers from the user.
4. Display a menu for different branching operations.
5. Read the user's choice.
6. Use a switch statement to select the required operation.
7. Perform the selected operation using branching statements:
   - Check whether a number is positive, negative, or zero.
   - Check whether a number is even or odd.
   - Find the largest of two numbers.
   - Check whether a number is divisible by 5.
8. Store the result in res wherever required.
9. Display the result.
10. If the user enters an invalid choice, display an appropriate message.
11. Stop the program.

PROGRAM:

```
#include <stdio.h>

int main()
{
   int a, b, choice, res;

   printf("===== BRANCHING STATEMENTS =====\n");

   printf("Enter the first number: ");
   scanf("%d", &a);

   printf("Enter the second number: ");
   scanf("%d", &b);

   printf("\n----- MENU -----\n");
   printf("1. Check Positive, Negative or Zero\n");
   printf("2. Check Even or Odd\n");
   printf("3. Find Largest of Two Numbers\n");
   printf("4. Check Divisibility by 5\n");

   printf("\nEnter your choice: ");
```

```
scanf("%d", &choice);

printf("\n----- RESULT -----\n");

switch(choice)
{
   case 1:
      if(a > 0)
         printf("%d is Positive", a);
      else if(a < 0)
         printf("%d is Negative", a);
      else
         printf("%d is Zero", a);
      break;

   case 2:
      if(a % 2 == 0)
         printf("%d is Even", a);
      else
         printf("%d is Odd", a);
      break;

   case 3:
      if(a > b)
      {
         res = a;
         printf("%d is the Largest Number", res);
      }
      else if(b > a)
      {
         res = b;
         printf("%d is the Largest Number", res);
      }
      else
      {
         printf("Both numbers are Equal");
      }
      break;

   case 4:
      if(a % 5 == 0)
         printf("%d is Divisible by 5", a);
      else
         printf("%d is Not Divisible by 5", a);
      break;

   default:
```

```
            printf("Invalid choice.");
        }

        return 0;
}
```

OUTPUT:

```
===== BRANCHING STATEMENTS =====
Enter the first number: 25
Enter the second number: 18

----- MENU -----
1. Check Positive, Negative or Zero
2. Check Even or Odd
3. Find Largest of Two Numbers
4. Check Divisibility by 5

Enter your choice: 3

----- RESULT -----
25 is the Largest Number
```

RESULT:

The C program for performing the use of branching statements (decision-making) has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
CO2	3	3	3	3	2	-	-	1	1	-	2		3	2	1

Ex No: 4	**DEVELOP PROGRAMS USING LOOPING STATEMENTS (ITERATION)**

AIM

To write a C program to demonstrates the use of Develop programs using looping statements.

ALGORITHM

1. Start the program.
2. Declare the integer variables n, i, sum, and res.
3. Read the value of n from the user.
4. Use a for loop to display the numbers from 1 to n.
5. Initialize sum to 0.
6. Use a while loop to calculate the sum of numbers from 1 to n.
7. Initialize res to 1.
8. Use a do-while loop to calculate the factorial of n.
9. Display the numbers, sum, and factorial.
10. Stop the program..

PROGRAM:

```
#include <stdio.h>
int main()
{
   int n, i, sum, res;
   printf("Enter a number: ");
   scanf("%d", &n);
   printf("\nNumbers from 1 to %d:\n", n);
   for(i = 1; i <= n; i++)
   {
      printf("%d ", i);
   }
   sum = 0;
   i = 1;
   while(i <= n)
   {
      sum = sum + i;
      i++;
   }

   res = 1;
   i = 1;
   do
   {
      res = res * i;
      i++;
   } while(i <= n);
   printf("\n\nSum of numbers = %d", sum);
   printf("\nFactorial of %d = %d", n, res);
```

```
    return 0;
}
```

OUTPUT:

```
Enter a number: 5

Numbers from 1 to 5:
1 2 3 4 5

Sum of numbers = 15
Factorial of 5 = 120
```

RESULT

The C program for performing the use of iterative statements (looping) has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**	**PSO 1**	**PSO 2**	**PSO 3**
CO2	3	3	3	3	2	-	-	1	1	-	2		3	2	1

Ex No: 5	DEMONSTRATE ARRAY OPERATIONS USING 1D AND 2D ARRAYS

AIM

To write a C program to demonstrate array operations using 1D and 2D arrays.

ALGORITHM

1. Start the program.
2. Declare a one-dimensional array a and a two-dimensional array b.
3. Read the number of elements for the 1D array.
4. Read the elements into the 1D array.
5. Display the elements of the 1D array.
6. Find the sum of the elements in the 1D array.
7. Read the number of rows and columns for the 2D array.
8. Read the elements into the 2D array.
9. Display the elements of the 2D array in matrix form.
10. Find the sum of all elements in the 2D array.
11. Display the results.
12. Stop the program

PROGRAM:

```
#include <stdio.h>
int main()
{
   int a[50], b[10][10];
   int n, rows, cols;
   int i, j, sum1, sum2;
   sum1 = 0;
   sum2 = 0;
   printf("Enter the number of elements in 1D array: ");
   scanf("%d", &n);
   printf("Enter %d elements:\n", n);
   for(i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }
   printf("\n1D Array Elements:\n");
   for(i = 0; i < n; i++)
   {
      printf("%d ", a[i]);
      sum1 = sum1 + a[i];
   }
   printf("\nSum of 1D Array = %d", sum1);
   printf("\n\nEnter the number of rows in 2D array: ");
   scanf("%d", &rows);
   printf("Enter the number of columns in 2D array: ");
   scanf("%d", &cols);
```

```
    printf("Enter the elements of 2D array:\n");
    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            scanf("%d", &b[i][j]);
        }
    }
    printf("\n2D Array Elements:\n");
    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            printf("%d\t", b[i][j]);
            sum2 = sum2 + b[i][j];
        }
        printf("\n");
    }
    printf("Sum of 2D Array = %d", sum2);
    return 0;
}
```

OUTPUT

```
Enter the number of elements in 1D array: 5
Enter 5 elements:
10
20
30
40
50

1D Array Elements:
10 20 30 40 50
Sum of 1D Array = 150

Enter the number of rows in 2D array: 2
Enter the number of columns in 2D array: 3
Enter the elements of 2D array:
1
2
3
4
5
6

2D Array Elements:
1   2       3
```

4 5 6
Sum of 2D Array = 21

RESULT

The C program for performing the use of array operations has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**	**PSO 1**	**PSO 2**	**PSO 3**
CO3	3	3	3	3	2	-	-	-	1	-	2		3	2	1

Ex No: 6	IMPLEMENT FUNCTIONS AND RECURSIVE SOLUTIONS

AIM

To write a C program for Implementing a functions and recursive solutions.

ALGORITHM

1. Start the program.
2. Declare the required variables and function prototypes.
3. Read an integer n from the user.
4. Call a user-defined function to find the square of n.
5. Call a recursive function to find the factorial of n.
6. In the recursive function, if n is 0 or 1, return 1.
7. Otherwise, return n * factorial(n - 1).
8. Store the returned values in res and fact.
9. Display the results.
10. Stop the program.

PROGRAM

```
#include <stdio.h>
int square(int n);
int factorial(int n);
int main()
{
   int n, res, fact;
   printf("Enter a number: ");
   scanf("%d", &n);
   res = square(n);
   fact = factorial(n);
   printf("\nSquare of %d = %d", n, res);
   printf("\nFactorial of %d = %d", n, fact);
   return 0;
}
int square(int n)
{
   int res;
   res = n * n;
   return res;
}
int factorial(int n)
{
   int res;
   if(n == 0 || n == 1)
   {
      return 1;
   }
```

```
    else
    {
        res = n * factorial(n - 1);
        return res;
    }
}
```

OUTPUT

Enter a number: 5

Square of 5 = 25
Factorial of 5 = 120

RESULT

The C program for performing the use of functions and recursive solutions has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**	**PSO 1**	**PSO 2**	**PSO 3**
CO3	3	3	3	3	2	-	-	-	1	-	2		3	2	1

Ex No: 7	C PROGRAMS USING STRINGS AND STRING FUNCTIONS

AIM

To write a C program using strings and string functions

ALGORITHM

1. Start the program.
2. Declare character arrays str1, str2, str3 and an integer variable res.
3. Read two strings from the user.
4. Find the length of the first string using strlen().
5. Copy the first string into the third string using strcpy().
6. Compare the two strings using strcmp().
7. Concatenate the second string with the first string using strcat().
8. Display the results of the string operations.
9. Stop the program.

PROGRAM

```
#include <stdio.h>
#include <string.h>
int main()
{
   char str1[50], str2[50], str3[100];
   int res;
   printf("Enter the first string: ");
   scanf("%s", str1);
   printf("Enter the second string: ");
   scanf("%s", str2);
   res = strlen(str1);
   printf("\nLength of first string = %d", res);
   strcpy(str3, str1);
   printf("\nCopied string = %s", str3);
   res = strcmp(str1, str2);
   if(res == 0)
      printf("\nBoth strings are equal");
   else
      printf("\nBoth strings are not equal");
   strcat(str1, str2);
   printf("\nConcatenated string = %s", str1);
   return 0;
}
```

OUTPUT

```
Enter the first string: Hello
Enter the second string: World
```

Length of first string = 5
Copied string = Hello
Both strings are not equal
Concatenated string = HelloWorld

RESULT

The C program for performing the use of C programs using strings and string functions has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**	**PSO 1**	**PSO 2**	**PSO 3**
CO3	3	3	3	3	2	-	-	-	1	-	2		3	2	1

Ex No: 8	POINTERS, PARAMETER PASSING TECHNIQUES AND DYNAMIC MEMORY ALLOCATION

AIM

To develop a C program to demonstrate the use of pointers, different parameter passing techniques, and dynamic memory allocation

ALGORITHM

1. Start the program.
2. Declare integer variables a, b, res, and a pointer ptr.
3. Read two integer values from the user.
4. Demonstrate pointer usage by storing the address of a in ptr.
5. Call a function using call by value to calculate the sum of two numbers.
6. Call another function using call by reference by passing the addresses of the variables.
7. Allocate memory dynamically using malloc().
8. Read the required number of elements into the dynamically allocated memory.
9. Calculate the sum of the dynamically allocated array elements.
10. Display the results.
11. Release the allocated memory using free().
12. Stop the program.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
/* Call by Value */
int addByValue(int a, int b)
{
   int res;
   res = a + b;
   return res;
}
/* Call by Reference */
void swapByReference(int *a, int *b)
{
   int temp;
   temp = *a;
   *a = *b;
   *b = temp;
}
int main()
{
   int a, b, res;
   int *ptr;
   int *arr;
   int n, i, sum;
   printf("Enter the first number: ");
```

```
    scanf("%d", &a);
    printf("Enter the second number: ");
    scanf("%d", &b);
    /* Pointer demonstration */
    ptr = &a;
    printf("\nPointer Demonstration:");
    printf("\nValue of a = %d", *ptr);
    printf("\nAddress of a = %p", (void *)ptr);
    /* Call by Value */
    res = addByValue(a, b);
    printf("\n\nCall by Value:");
    printf("\nSum = %d", res);
    /* Call by Reference */
    swapByReference(&a, &b);
    printf("\n\nCall by Reference:");
    printf("\nAfter swapping, first number = %d", a);
    printf("\nAfter swapping, second number = %d", b);
    /* Dynamic Memory Allocation */
    printf("\n\nDynamic Memory Allocation");
    printf("\nEnter the number of elements: ");
    scanf("%d", &n);
    arr = (int *)malloc(n * sizeof(int));
    if(arr == NULL)  {
       printf("Memory allocation failed.");
       return 0;
    }
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
       scanf("%d", &arr[i]);
    }
    sum = 0;
    for(i = 0; i < n; i++)
    {
       sum = sum + arr[i];
    }
    printf("Sum of dynamically allocated array = %d", sum);
    free(arr);
    return 0;
}
```

OUTPUT

```
Enter the first number: 10
Enter the second number: 20

Pointer Demonstration:
Value of a = 10
Address of a = 000000000061FDE4
```

Call by Value:
Sum = 30

Call by Reference:
After swapping, first number = 20
After swapping, second number = 10

Dynamic Memory Allocation
Enter the number of elements: 5
Enter 5 elements:
1
2
3
4
5
Sum of dynamically allocated array = 15

RESULT

The C program for performing the use of C programs using strings and string functions has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**	**PSO 1**	**PSO 2**	**PSO 3**
CO4	3	3	3	3	2	-	-	-	1	-	2		3	2	1

Ex No: 9	APPLY FILE HANDLING CONCEPTS ALONG WITH THE USE OF STRUCTURES AND UNIONS

AIM

To write a program to Generate the Fibonacci sequence up to a given number using functions.

ALGORITHM

1. Start the program.
2. Declare a structure Student to store student details.
3. Declare a union Contact to store either a phone number or an email.
4. Read the student details from the user.
5. Store the details in a structure variable.Store the contact information using the union.
6. Open a file in write mode using fopen().Write the student details into the file using fprintf().Close the file using fclose().
7. Open the same file in read mode.
8. Read and display the stored student details.
9. Close the file.
10. Stop the program.

PROGRAM

```
#include <stdio.h>
struct Student{
   int rollno;
   char name[50];
   float mark;
};
union Contact{
   char phone[15];
   char email[50];
};
int main() {
   struct Student s;
   union Contact c;
   FILE *fp;
   printf("Enter student roll number: ");  scanf("%d", &s.rollno);
   printf("Enter student name: "); scanf("%s", s.name);
   printf("Enter student mark: "); scanf("%f", &s.mark);
   printf("Enter phone number: "); scanf("%s", c.phone);
   fp = fopen("student.txt", "w");
   if(fp == NULL)    {
      printf("Unable to open file.");
      return 0;
   }
   fprintf(fp, "Roll Number : %d\n", s.rollno);
   fprintf(fp, "Name        : %s\n", s.name);
   fprintf(fp, "Mark        : %.2f\n", s.mark);
   fprintf(fp, "Phone       : %s\n", c.phone);
```

```
    fclose(fp);
    printf("\nStudent details written to file successfully.\n");
    fp = fopen("student.txt", "r");
    if(fp == NULL) {
        printf("Unable to open file.");
        return 0;
    }
    printf("\n----- STUDENT DETAILS -----\n");
    while(fscanf(fp, "%[^\n]%*c", s.name) == 1) {
        printf("%s\n", s.name);
    }
    fclose(fp);
    return 0;
}
```

OUTPUT

```
Enter student roll number: 101
Enter student name: Arun
Enter student mark: 87.5
Enter phone number: 9876543210

Student details written to file successfully.

----- STUDENT DETAILS -----
Roll Number : 101
Name        : Arun
Mark        : 87.50
Phone       : 9876543210
```

RESULT

The C program for performing the apply file handling concepts along with the use of structures and union has been executed successfully, and the output has been verified. By performing this experiment the following CO's and PO's are attained.

PO's CO's	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**	**PSO 1**	**PSO 2**	**PSO 3**
CO5	3	3	3	3	2	1	1	1	1	1	2		3	3	3

Ex No: 10	DEVELOP A MINI PROJECT FOR A REAL-TIME APPLICATION

AIM

To develop a mini project in C for managing student attendance in real time using structures, functions, arrays, and file handling

ALGORITHM

1. Start the program.
2. Define a structure Student containing roll number, name, and attendance status.
3. Read the number of students.
4. Read the roll number and name of each student.
5. Record the attendance status of each student as Present or Absent.
6. Store the attendance details in a file.
7. Display the current attendance details on the screen.
8. Calculate the number of present and absent students.
9. Display the attendance summary.
10. Close the file.
11. Stop the program.

PROGRAM

```
#include <stdio.h>
struct Student
{
   int rollno;
   char name[50];
   char attendance;
};
int main()
{
   struct Student s[50];
   FILE *fp;
   int n, i;
   int present = 0, absent = 0;
      printf("===== REAL-TIME ATTENDANCE SYSTEM =====\n");
   printf("Enter the number of students: ");
   scanf("%d", &n);
   printf("\nEnter student details:\n");
   for(i = 0; i < n; i++)
   {
      printf("\nStudent %d\n", i + 1);
      printf("Roll Number: ");
      scanf("%d", &s[i].rollno);
      printf("Name: ");
      scanf("%s", s[i].name);
      printf("Attendance (P/A): ");
      scanf(" %c", &s[i].attendance);
```

```
        if(s[i].attendance == 'P' || s[i].attendance == 'p')
        {
            present++;
        }
        else
        {
            absent++;
        }
    }
    fp = fopen("attendance.txt", "w");
    if(fp == NULL)
    {
        printf("Unable to open file.");
        return 0;
    }
    fprintf(fp, "===== ATTENDANCE REPORT =====\n");
    fprintf(fp, "Roll No\tName\tAttendance\n");
    for(i = 0; i < n; i++)
    {
        fprintf(fp, "%d\t%s\t%c\n",
                s[i].rollno,
                s[i].name,
                s[i].attendance);
    }
    fprintf(fp, "\nPresent = %d\n", present);
    fprintf(fp, "Absent = %d\n", absent);
    fclose(fp);
    printf("\n===== ATTENDANCE REPORT =====\n");
    printf("Roll No\tName\tAttendance\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\t%s\t%c\n",
                s[i].rollno,
                s[i].name,
                s[i].attendance);
    }
    printf("\nTotal Present = %d", present);
    printf("\nTotal Absent = %d", absent);
    printf("\n\nAttendance saved successfully.");
    return 0;
}
```

OUTPUT

```
===== REAL-TIME ATTENDANCE SYSTEM =====
Enter the number of students: 3
```

Enter student details:

Student 1
Roll Number: 101
Name: Arun
Attendance (P/A): P

Student 2
Roll Number: 102
Name: Kumar
Attendance (P/A): A

Student 3
Roll Number: 103
Name: Priya
Attendance (P/A): P

===== ATTENDANCE REPORT =====
Roll No Name Attendance
101 Arun P
102 Kumar A
103 Priya P

Total Present = 2
Total Absent = 1

Attendance saved successfully..

RESULT

The C program for developing a real-time Student Attendance Management System has been successfully implemented using structures, functions, arrays, and file handling concepts. The program was executed successfully, and the output was verified. By performing this experiment, the relevant COs and POs were attained.

PO's CO's	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**	**PSO 1**	**PSO 2**	**PSO 3**
CO5	3	3	3	3	2	1	1	1	1	1	2		3	3	3